

Reactive Keypad Final Critique

Miles, Brady, Eve, Christian, Evelyn



Project Description

Our project focuses on designing and building a form-changing keyboard that can physically reshape its surface in real time. The keyboard is constructed from an array of small, independently actuated square modules that move up and down. By precisely controlling these modules, the table can dynamically create 3D surfaces, replicate objects placed on it, or adapt to different user needs.

Final Product

For our final product this semester, we created a 3×3 “proof of concept” array that would function as a modular section of the keypad/table.

This module, including the extra wiring necessary for lighting and keys, is housed in a box made out of black acrylic that contains an extra button acting as an emergency stop.

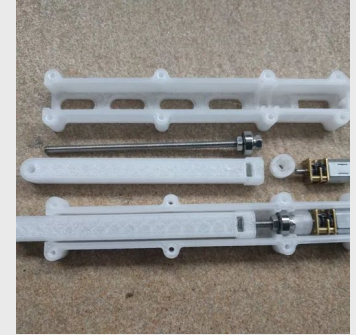
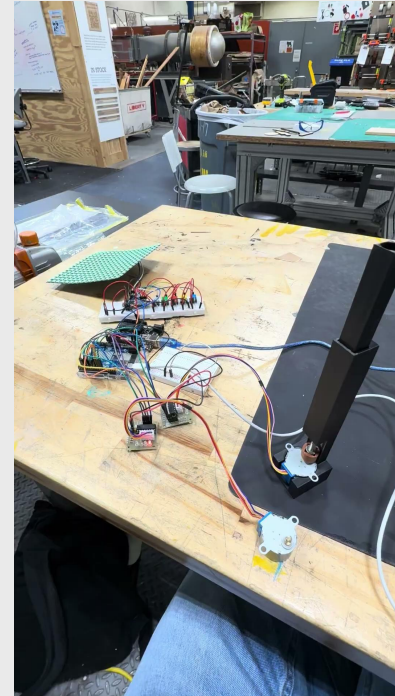
Also attached to the module is a lighting array that is currently customizable using keys situated within the acrylic box.





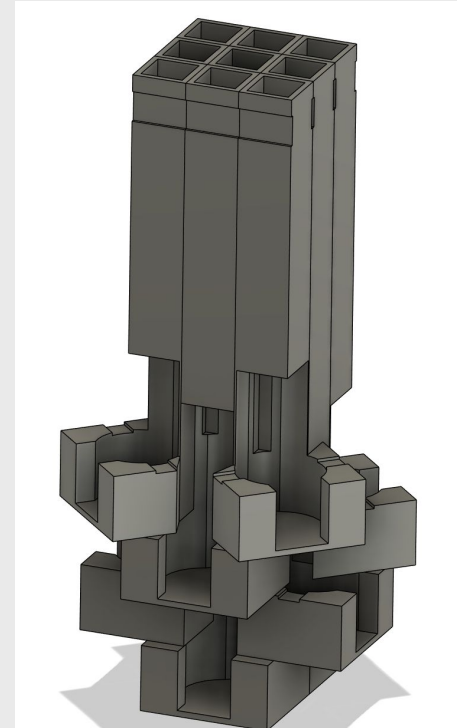
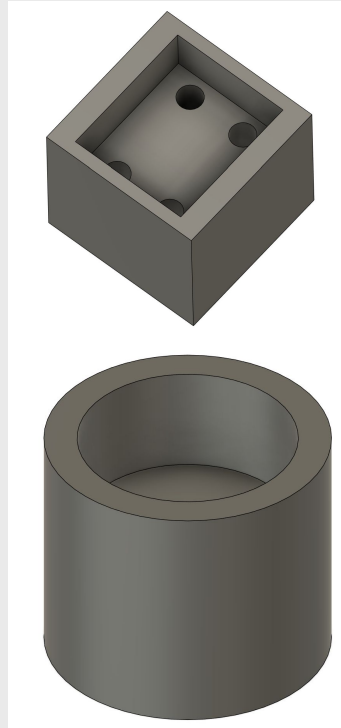
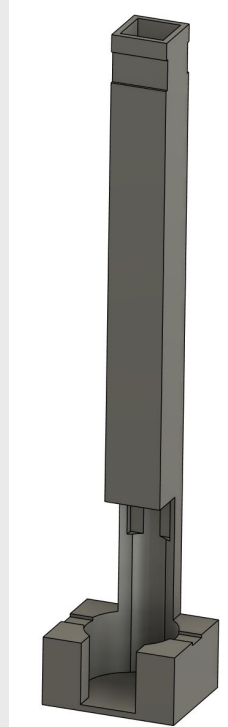
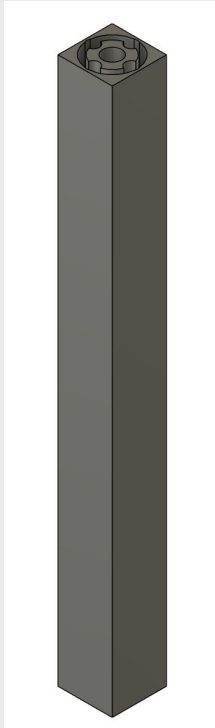
Iterative Prototyping

- Initial prototypes of keypad included:
 - IO expanders with Arduino Uno
 - Raspberry Pi with serial communication to the arduino
 - 1-2 Stepper motors in operation
 - LCD Screen to Display Current Game Status
 - LED Array to display colors
 - Testing of threaded DC motors
- Raspberry Pi communication did work, but would require more complex programming. With the Pi, there could be independence from the computer and custom animations





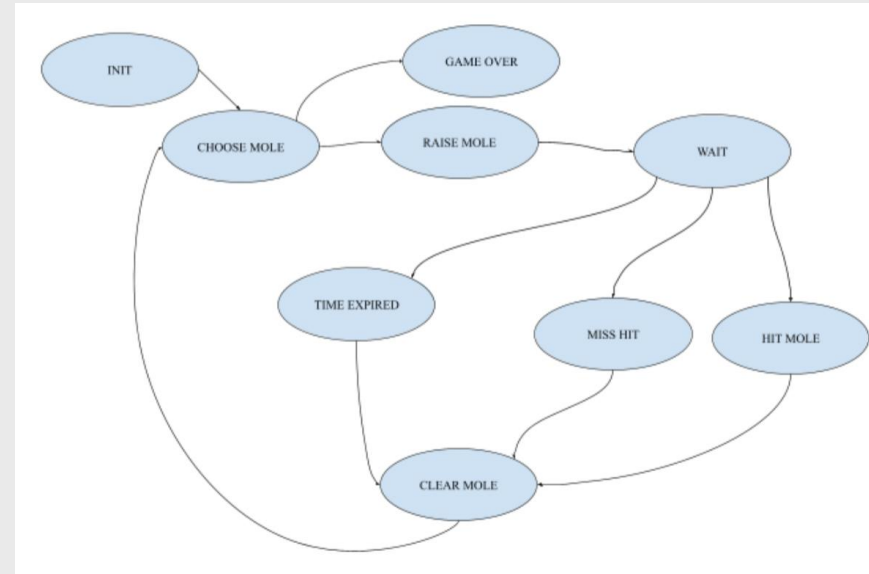
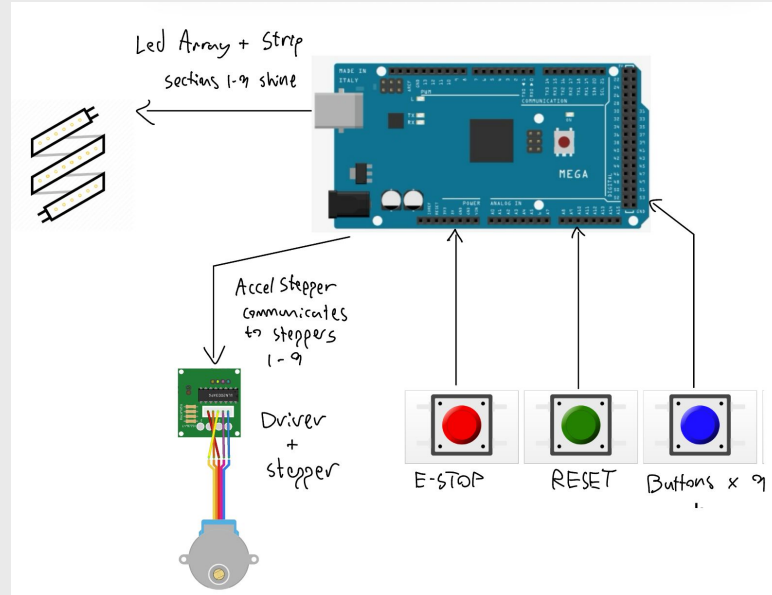
Actuator CAD





Wiring and Programming

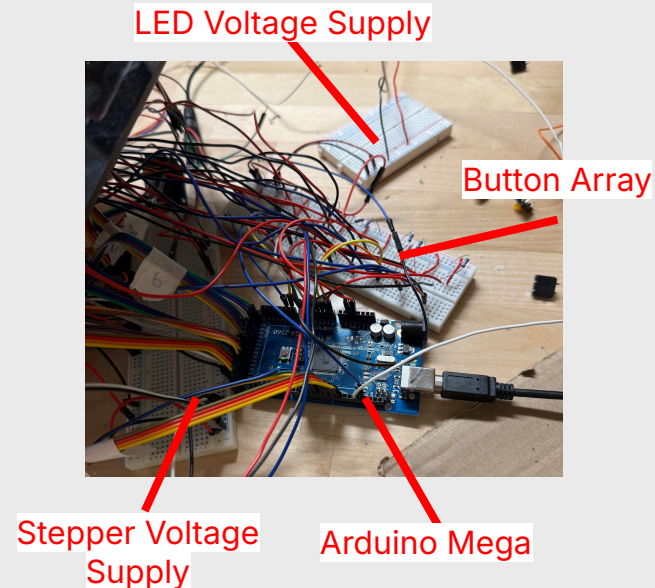
- Final Prototype Plays Whack a Mole using Finite State Machine





Wiring and Programming

- Electronic components of the final keypad prototype include:
 - 9 × 28byj-48 stepper motors with drivers
 - Arduino mega (Arduino uno with io expanders in initial prototypes)
 - Digital indexing LED Array and strip
 - 10 x mechanical buttons including emergency stop
 - Serial communication with user computer to provide inputs to game and display output messages
 - Emergency Stop and Reset Buttons for safety
- System requires 12V, 2A input for stepper motors, which each require approximately 200mA and 5-12V that is regulated by the stepper motor drivers
- Transition to arduino Mega motivated by need to increase number of digital outputs so that stepper motors could move more quickly and smoothly



Engineering Standards and Testing

Engineering standards were derived from the United States’ Federal Aviation Administration Human Factors in Design and Safety (FAA HFDS) standards for keyboard use. While these do not represent the industry standard globally, they provide a stepping point for keyboard design and layout. Future engineering standards to be used include ISO/IEC 9995 and ASME’s “Force Feedback Control for Manipulator Fine Motions.”

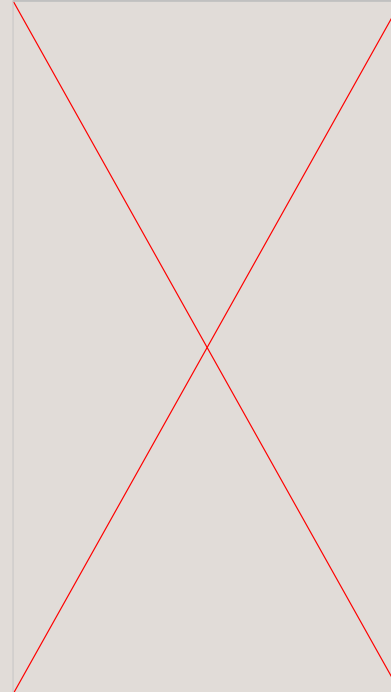
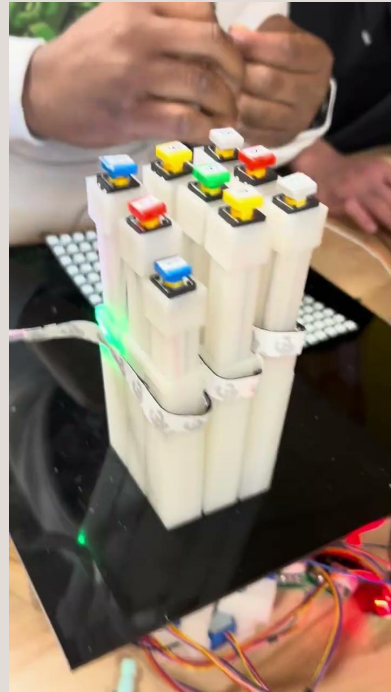
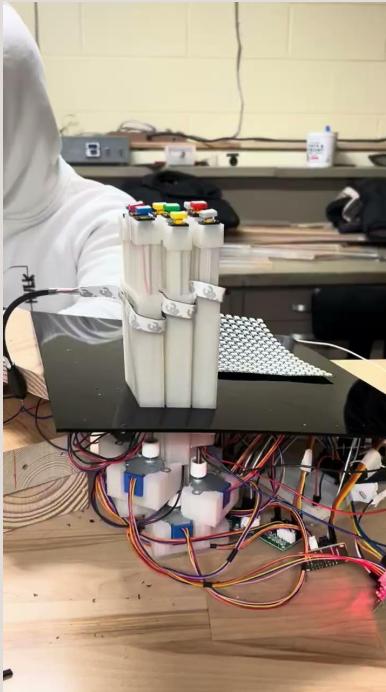
1. Key Dimensions Individual keys do not exceed 18 mm by 18 mm.	2. Slope Angle Keyboard slope does not exceed 15 degrees.	3. Key Layout Keys follow the standard QWERTY or number pad layout.	4. Key Shape The actuator caps and buttons are square-shaped.
5. Key Curvature The button tops/keycaps are concave.	6. Legibility Keyboards are free from reflection and have easily legible characters.	7. Label Contrast Keypad label luminance contrast ratio is at minimum 3:1.	8. Depression Force Force to depress keys is between 0.25 and 1.5 N. (0.9 and 5.3 oz)

Product Demonstration Videos

Description and Applications

In this demonstration video, the keypad is shown to be functional as a regular keypad, the lights are shown to be functional, and the actuators are shown to be operational independent from each other.

Applications of a tactile force feedback keypad include common, everyday keyboard use, video gaming, augmented reality, and lessons in learning how to type.



Capstone Critique	ENGN1000	12/8/2025	
Complications			
Motor Acquisition	The main design discussions revolved around choosing between DC and stepper motors. Even the final week before the prototype had to be made, we were still debating whether to use DC motors at the cost of precision or steppers at the cost of size, a big ask in a project like this where a small size is preferred. Most, if not all, motors were also expensive and would not arrive on time.	DC	DC (Direct Current) Motors: Smaller and more cost-efficient, but in need of a separate tuning system for precision. Eventually decided against for the time being.
		S	Stepper motors: Generally larger and more expensive, but much more precise. The stepper motors used in the final product were also discovered to be prone to overheating during use.
Manufacturing Plan	Without a manufacturing plan, assembly was complicated, and the assembly process had to be repeated many times. Additionally, manufacturing pieces such as the connector between the stepper motors and the lead screw without the final table assembly dimensions easily accessible resulted in preventable spacing issues that took up extra time and effort to fix.		
3D Printing	During the week of assembly the main thing holding us back was how slow printing all the pieces was. The CAD had to be fine-tuned several times to accommodate not just changes in design but also initial oversights like the pathways used to help straighten the wires through the pillars. Even when the final design was chosen, the prints themselves often failed overnight if we chose to print several pieces at a time, forcing us to take longer.	1	Purchase multiple spools of filament instead of just one, to get more prints running at the same time.
		2	Accept single prints and take the hit on time despite the rushed schedule.

Design Impacts



Economic

Estimates vary, but the global keyboard market is valued from hundred of millions to multiple billion dollars USD. A significant portion of this market is due to mechanical keyboards, which are extremely popular among the video gaming community.



Social

An interactive keyboard would provide more opportunities for augmented reality in video gaming experiences. Implementation would have implications for both the entertainment community and in educational settings, as the adjustable heights of keys allow for typing education games.



Environmental

While this design was not created with sustainability in mind, it has the potential to be manufactured from primarily sustainable materials, and requires minimal energy. Future iterations will further explore the impact that this product has on the environment, and how to combat that.



Global

Keypads and keyboards are used across the globe for day-to-day office work, video gaming, communication, and more. While keyboard layouts change depending on the country/language, a product such as the reactive keypad has the ability to be implemented throughout the world.

Future Iterations and Goals

Other future goals:	Raspberry Pi Button Encoding	Smaller, Faster Motors	Set Manufacturing Plans
- Attendance and demonstration at a IEEE conference - Paper/article published detailing the manufacturing process, use, and applicability of a reactive keypad - Completion of ENG1001, the spring capstone course, as a continuation of this semester's work	As we continue to expand the number of keys on the table, we will connect the buttons to a master Raspberry Pi that works through serial communication with several Arduinos at a time. This will enable us to expand the number of I/O ports available and the number of functional keys as a result. The code itself would just detect positive button edges (if we take the buttons as a digital signal) and encode them as a series of booleans in a string. This encoded list would then get sent in real time to the Arduinos at a high rate (every 0.01s or faster). The code can also be changed if it's preferable to read and send only the <i>current</i> state of all buttons at all times depending on the purpose of the program.	Due to the larger size of the stepper motors used in the assembly, the linear actuators had to be assembled in a stacked configuration. While this stacked configuration worked for a temporary, smaller module, it is not a feasible solution for a larger keypad or table design. As such, smaller stepper motors that can fit within an actuator column are necessary to maintain the usability and manufacturability of the keypad itself. Additionally, the current stepper motors move at a rate of 4000 SPM (steps per minute), which only translates to 40 RPM. The new motors will have a higher RPM in order to raise the actuator faster.	While all of the actuator pieces fit together individually, slotting them together into the module configuration proved to be more difficult than expected. A manufacturing plan created with modular assembly in mind would ease the burden of trying to fit all of the pieces of the keypad together while keeping them steady. Additionally, the wiring connecting all the different pieces (power, motors, drivers, breadboards, Pi, arduinos, etc.) has proved to be very complicated, and a manufacturing plan would provide a means to contain the wiring in an organized way.

Raspberry Pi Code for Button Detection

```
import serial
import time
from gpiozero import Button

ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1.0)
time.sleep(3)
ser.reset_input_buffer()
print("Serial OK")

buttons = [Button(23), Button(24), Button(25)]
previous_states = [False] * len(buttons)
real_states = [False] * len(buttons)

while True:
    for i, button in enumerate(buttons):
        current_state = not button.is_pressed

        if previous_states[i] and not current_state:
            real_states[i] = not real_states[i]

        previous_states[i] = current_state

    parsed_real_states = (''.join(['1' if s else '0' for s in real_states])) + "\n"

    ser.write(parsed_real_states.encode('utf-8'))

    time.sleep(0.01)
```

Description

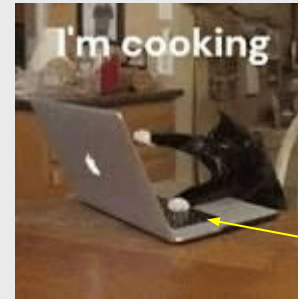
In the future, this code could be used to read button presses and enable us to expand the number of I/O ports to create a larger keyboard/table.

This code works by initializing the buttons to an off state and then comparing the current state of all buttons against their previous state in order to detect if a positive edge has been triggered and the button has been pressed. The list of 'real states' is then parsed, encoded, and sent to the Arduino over a direct USB serial connection link.

One thing to notice is that the logic is inverted because the buttons were wired/tested like that (pressed = off and vice versa). The code is functional and has been tested.



Thank you!



Keypad (reactive)